

How Agent A Works

A visual walkthrough of the Curriculum Architect — one flowchart and one analogy that explain the whole system, no code reading required.

In one sentence

Agent A does exactly one job:

It receives a **course order** (which subject, what age group, what topic) and outputs a **course blueprint** — how many modules, what each module teaches, which comes before which, and how long each session runs. It **never writes teaching content**; that is Agent B's job. Agent A only draws the skeleton.

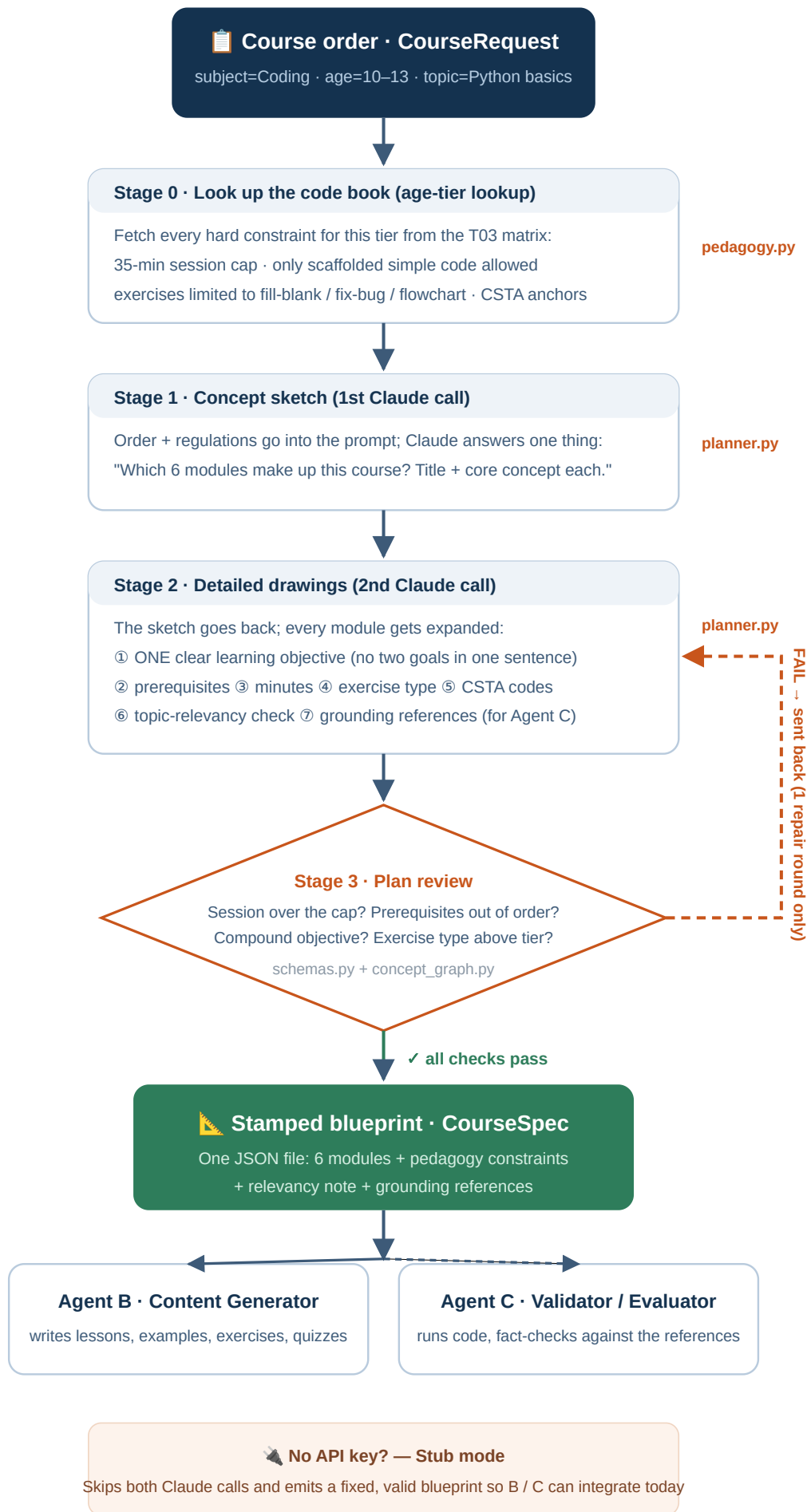
The analogy: an architecture firm

The agent is literally called an "architect", and the whole system maps cleanly onto how an architecture firm works:

AT THE ARCHITECTURE FIRM	INSIDE AGENT A
The client's commission form	CourseRequest — subject + age bracket + topic. Deliberately just those three things.
The national building code	pedagogy.py — your T03 pedagogy matrix. "No session over 20 minutes for ages 6–9, no raw code ever" is a hard regulation , not a suggestion — exactly like "ceiling height must be at least 2.8 m".
Structural sequencing rules (foundations before floors)	concept_graph.py — the prerequisite concept graph. "Loops" must come after "conditionals", just as the third floor cannot be built before the second.
Concept sketch first	Stage 1 — Claude is asked for the module list only (titles + concepts), no detail yet.
Then detailed construction drawings	Stage 2 — Claude expands every module: learning objective, prerequisites, minutes, exercise type.
Plan-review office inspects before stamping; failures sent back once for revision	Stage 3 — the rule validator checks the blueprint against every regulation; violations are sent back to Claude for exactly one repair round.
The stamped, official drawings	CourseSpec (a JSON file) — handed to the construction crew (Agent B) to build the actual content.
Template/sample drawings	Stub mode — with no API key, the system still emits a perfectly formatted dummy blueprint so teammates B and C can integrate and test today.

The full flowchart

The complete path data travels, from incoming order to delivered blueprint:



File map: what each file does

The whole project is five core files, each with one clear responsibility:

schemas.py

The contract / official form

Defines what an "order" and a "blueprint" must look like. Think of a government form: which fields are mandatory, which number ranges are legal. Bad input is rejected on the spot — schedule a 90-minute session for a 10–13 course and this file throws an error immediately. It is also the team's **shared data contract**: changing any field requires sign-off from all three of you.

pedagogy.py

The building-code book

Your T03 pedagogy matrix, converted from a Word document into a data table in code. One column per age tier: cognitive theory, CSTA level, whether raw code is allowed, session caps... **This is where your research lands in the system** — and the dissertation's evidence for "how constraints enter the planning process".

concept_graph.py

Construction sequencing rules

A hand-curated dependency graph of Python concepts: "conditionals" before "loops", "loops" before "functions". Whatever ordering Claude proposes is checked against this graph — out-of-order equals violation.

planner.py

The chief engineer (core)

The brain that runs the whole pipeline: look up regulations → two Claude calls (sketch, then detail) → send to review → one repair round on failure → issue the blueprint. Both prompts live here too — the prompts are your "instructions for directing Claude", key material for the methodology chapter.

tests/test_agent_a.py

The QC checklist (13 checks)

Thirteen automated checks: do all four tiers produce valid blueprints? Do ages 6–9 really never exceed 20 minutes? Does a deliberately wrong ordering get caught? One command runs them all — exactly the "test each agent before handoff" requirement from both the supervisor and the company.

What the final output looks like

All of Agent A's work condenses into one JSON file (simplified):

```
{
  "course_id": "coding_creators_5c414e",
  "age_bracket": "Creators (10-13)",           ← who the course is for
  "pedagogy": {
    "max_session_min": 35,                     ← hard constraints from the code book
    "raw_text_code_allowed": "limited",
    "exercise_types": ["fill_in_the_blank", ...]
  },
  "relevancy_note": "...",                     ← is the topic still current (2 Jul meeting)
  "references": [ CSTA standard links... ],    ← grounding sources, verified by Agent C
  "modules": [
    { "module_id": "m1", "objective": "Understand what a program is",
      "prerequisites": [], "target_minutes": 30 },
    { "module_id": "m2", "objective": "Understand sequential instructions",
      "prerequisites": ["m1"], ... }           ← m2 builds on m1
  ]
}
```

Agent B fills each module with actual teaching content. The pedagogy block is injected verbatim into B's prompt — so B can never assign a "code review" exercise to a nine-year-old.

The three points most valuable for your dissertation

① **How constraints are encoded into planning** — the direct answer to your research question: the pedagogy matrix becomes a data table, the prerequisite graph becomes a directed graph, and both are injected into prompts as hard constraints then enforced by a validator. This is where the supervisor's "why these inputs, why these parameters" gets answered.

② **Quantifiable performance measures** — the plan-review stage naturally produces numbers: prerequisite-violation rate, session-cap satisfaction rate, compound-objective rate, one-round repair success rate. Your (mathematician) supervisor's demand for formal performance measures is distilled from here.

③ **Hierarchical vs. single-shot planning** — the "sketch first, then detail" two-stage design is itself an experimental variable: same order, one-shot generation vs. two-stage generation — which yields fewer violations? That comparison can become a small experiment in your dissertation.